

QUIZ NOTES

ICLEI ARTHUR RODRIGUES DE OLIVEIRA

CARLOS EDUARDO DE LIRA ALMEIDA

THALITA IZABELLE CUNHA DOS SANTOS SILVA

GUILHERME BATISTA ALVES

JOÃO MIGUEL FRANCISCO DE SOUZA

DESENVOLVIMENTO DE APLICAÇÃO MOBILE EM FLUTTER

Trabalho apresentado à disciplina de
Desenvolvimento para Dispositivos Móveis
como requisito para obtenção de nota.
Professor: Italo Nunes Pereira

SUMÁRIO

1 INTRODUÇÃO	4
1.1 Contextualização	4
1.2 Objetivo Geral	4
1.3 Objetivos Específicos	4
1.4 Escopo	5
2 REFERENCIAL TEÓRICO	5
2.1 Qualidade de Software	5
2.2 Ciclo de Vida de Software	6
2.3 Maturidade de Software	7
3 DESCRIÇÃO DO SISTEMA	7
3.1 Visão Geral	7
3.2 Funcionalidades	8
4 REQUISITOS DO SISTEMA	8
4.1 Requisitos Funcionais	8
4.2 Requisitos Não Funcionais	8
5 MODELAGEM DO SISTEMA	9
5.1 Casos de Uso	9
5.2 Diagrama de Classes	10
5.3 Banco de Dados	10
6 ARQUITETURA DE SOFTWARE	12
7 PROCESSO DE DESENVOLVIMENTO	13
8 GARANTIA DA QUALIDADE	14
8.1 Critérios de Qualidade	14
8.2 Testes	14
9 AVALIAÇÃO DE MATURIDADE	14
10 GERENCIAMENTO DE CONFIGURAÇÃO	15
11 SEGURANÇA DA INFORMAÇÃO	16
12 IMPLANTAÇÃO	16
13 MANUTENÇÃO FINAL	16
14 CONSIDERAÇÕES FINAIS	17
REFERÊNCIAS	18

1 INTRODUÇÃO

1.1 Contextualização

Através da disciplina de Desenvolvimento para Dispositivos Móveis, foram atribuídas as responsabilidades de desenvolver um sistema e sua respectiva documentação. O projeto consiste em desenvolver um aplicativo de estudos através de flashcards, eliminando a necessidade de escrever resumos grandes e verbosos para assim resumi-los em pequenos quizzes com frente e verso. O sistema se torna relevante, tendo em vista o crescimento da tecnologia e alta demanda por conteúdo informativo através da internet. Assim, nosso aplicativo torna mais simples o ato de estudar e revisar.

Os aplicativos são ótimas ferramentas para quem estuda sozinho, estimulando o estudante a se desafiar, realizando uma espécie de jogo de adivinhação com a máquina. Essas práticas fazem parte do que é conhecido como “estudo ativo”. Ao estudar ativamente, a pessoa percebe uma melhora significativa na compreensão e retenção do conteúdo aprendido. Estudar ativamente consiste em responder questões, preparar resumos, elaborar mapas mentais e até mesmo explicar o conteúdo a si mesmo.

Além de ser uma ferramenta muito poderosa para autodidatas, cremos que se bem implementada, o sistema pode compor o método de aprendizagem de diversas instituições de ensino, tendo em vista o seu alto nível de eficácia.

1.2 Objetivo Geral

O objetivo ao desenvolver esse sistema é possibilitar uma maneira mais fácil, rápida e otimizada de estudar e revisar os mais diversos assuntos, assim, facilitando o processo de aprendizagem e da retenção das partes mais relevantes dos conteúdos.

1.3 Objetivos Específicos

- Desenvolver um aplicativo mobile funcional
- Aplicar boas práticas de engenharia de software
- Garantir qualidade baseada em normas internacionais

- Desenvolver uma interface amigável e intuitiva de se utilizar
- Aplicar testes unitários para garantia de qualidade de software

1.4 Escopo

Farão parte do escopo: Uma seção para adicionar, modificar e deletar cards; uma seção para organizar cards em baralhos (contexto); um algoritmo para resgatar os cards criados pelo usuário em períodos de tempo baseado na performance do estudante(usuário); uma funcionalidade que permite o usuário revisar os cards sempre que ele clicar no baralho respectivo; uma seção principal onde serão organizados todos os baralhos; uma informativo do desempenho do usuário em um baralho ao clicar num botão. O projeto se diz respeito a tudo que foi definido no escopo. Aquilo que for solicitado, e não estiver definido no escopo, poderá ser desenvolvido em um novo projeto.

2 REFERENCIAL TEÓRICO

2.1 Qualidade de Software

Baseado na norma ISO/IEC 25010, a qualidade de software é avaliada por:

- Funcionalidade: O aplicativo irá permitir o usuário criar, editar, excluir e manipular cards e baralhos; será possível revisar os cards sobre um algoritmo de repetição espaçada; o desempenho do estudante será calculado e exibido para ele.
- Confiabilidade: O sistema demonstra confiabilidade ao garantir a persistência dos dados em um banco de dados relacional e ao manter sua estabilidade mesmo durante longas sessões de estudo, minimizando possíveis falhas de sincronização.
- Usabilidade: No sistema, a usabilidade é garantida por uma interface intuitiva, facilitando a curva de aprendizado do usuário e permitindo que ele opere as funções de revisão sem a necessidade de realizar um treinamento complexo.
- Eficiência: A eficiência de desempenho é assegurada pela renderização otimizada do Flutter, que proporciona respostas rápidas às interações do

usuário e baixo consumo de memória e bateria, sendo adequado para dispositivos mobile com hardware de entrada.

- **Manutenibilidade:** Ao utilizar ferramentas como Git e Flutter, por serem modulares, isso facilita o trabalho de realizar manutenções ou adicionar novas funcionalidades que surgem ao decorrer do desenvolvimento.
- **Portabilidade:** Ao utilizar um framework multiplataforma como o Flutter, fica mais simples portar o sistema tanto para IOS, quanto para Android, embora o foco principal da aplicação seja Android.

2.2 Ciclo de Vida de Software

Baseado na ISO/IEC 12207, o ciclo inclui:

- **Levantamento de requisitos:** Nós encontramos a necessidade de desenvolver uma ferramenta de memorização baseado no quanto um assunto foi compreendido pelo usuário. Definimos que o sistema precisava ser simples e intuitivo para que o usuário consiga utiliza-lo sem muitas dificuldades; o usuário precisava ter a liberdade de criar e personalizar os seus cards da maneira que precisasse. Assim, desenvolvemos a funcionalidade de criar e modificar cards, acessada através de botões.
- **Desenvolvimento:** O desenvolvimento passou a ser dividido em duas frentes modulares: o aplicativo mobile multiplataforma em Flutter baseado no padrão MVC e a API RESTful em Node.js com Express e TypeScript. A implementação do backend segue rigorosamente a metodologia TDD (Test-Driven Development), aplicando o ciclo Red-Green-Refactor para garantir que o código de produção seja escrito estritamente para satisfazer os requisitos de testes previamente definidos. O versionamento de todo o ecossistema é gerido via Git e o fluxo GitFlow, isolando as implementações e correções em branches específicas.
- **Testes:** No backend, o framework Jest é utilizado para executar testes unitários automatizados (isolando a lógica de negócio e o algoritmo de repetição) e testes de integração (validando o comportamento das rotas Express e a

persistência no Supabase). No frontend, mantêm-se os testes funcionais e de usabilidade manuais em emulador Android para garantir a qualidade da interface com o utilizador.

- **Implantação:** A entrega final consiste na disponibilização do código-fonte via repositório no GitHub e na geração de um executável (APK) para instalação direta em dispositivos Android, além do deploy da API Node.js em ambiente de nuvem, e configuração das regras de acesso no Supabase.
- **Manutenção:** A manutenibilidade é muito favorecida pelo desacoplamento entre o cliente e o servidor. A arquitetura em camadas permite a execução de manutenções corretivas, adaptativas e evolutivas de forma isolada. Desta forma, correções de regras de negócio ou otimizações na API em TypeScript podem ser aplicadas diretamente no backend, sem a necessidade de forçar o utilizador a efetuar a atualização do aplicativo Android no dispositivo móvel.

2.3 Maturidade de Software

Considerando a natureza do Quiz Notes como um Produto Mínimo Viável (MVP) desenvolvido em um contexto acadêmico, o projeto se enquadra predominantemente no Nível 1 – Inicial.

Apesar de estar no nível inicial, o projeto aplica práticas fundamentais que visam a transição para o Nível 2 – Gerenciado. Entre essas práticas, destacam-se a gestão de requisitos baseada no escopo definido e a forte adoção de ferramentas de controle de configuração, como o uso do versionamento em Git e a organização de ramificações através do GitFlow. O objetivo prioritário é validar as funcionalidades essenciais do aplicativo antes de implementar métricas e processos institucionais mais complexos.

3 DESCRIÇÃO DO SISTEMA

3.1 Visão Geral

- Nome do sistema: Quiz Notes
- Plataforma: Android

- Público-alvo: Estudantes e Professores

3.2 Funcionalidades

Adicionar, modificar e deletar cards e baralhos; um algoritmo de revisão espaçada com o tempo baseado na performance do usuário; opção de revisão livre ao clicar no respectivo baralho requisitado; organizar cards em baralhos. Calcular o desempenho do usuário baseado nas revisões dos cards.

4 REQUISITOS DO SISTEMA

4.1 Requisitos Funcionais

- RF01 - Gerenciamento de Baralhos: O sistema deve permitir ao usuário criar, modificar, listar e deletar baralhos para organizar seus estudos.
- RF02 - Gerenciamento de Cards: O sistema deve permitir ao usuário adicionar, modificar e deletar cards (com frente e verso) dentro de um baralho específico.
- RF03 - Revisão Espaçada: O sistema deve possuir um algoritmo que apresente os cards para revisão em intervalos de tempo calculados com base na performance anterior do usuário.
- RF04 – Revisão Livre: O sistema deve permitir que o usuário realize revisões livres de um baralho a qualquer momento, independente do algoritmo de repetição.
- RF05 – Relatório de Desempenho: O sistema deve calcular e exibir o desempenho do usuário baseado no seu histórico de revisões dos cards.

4.2 Requisitos Não Funcionais

Baseados na ISO/IEC 25010:

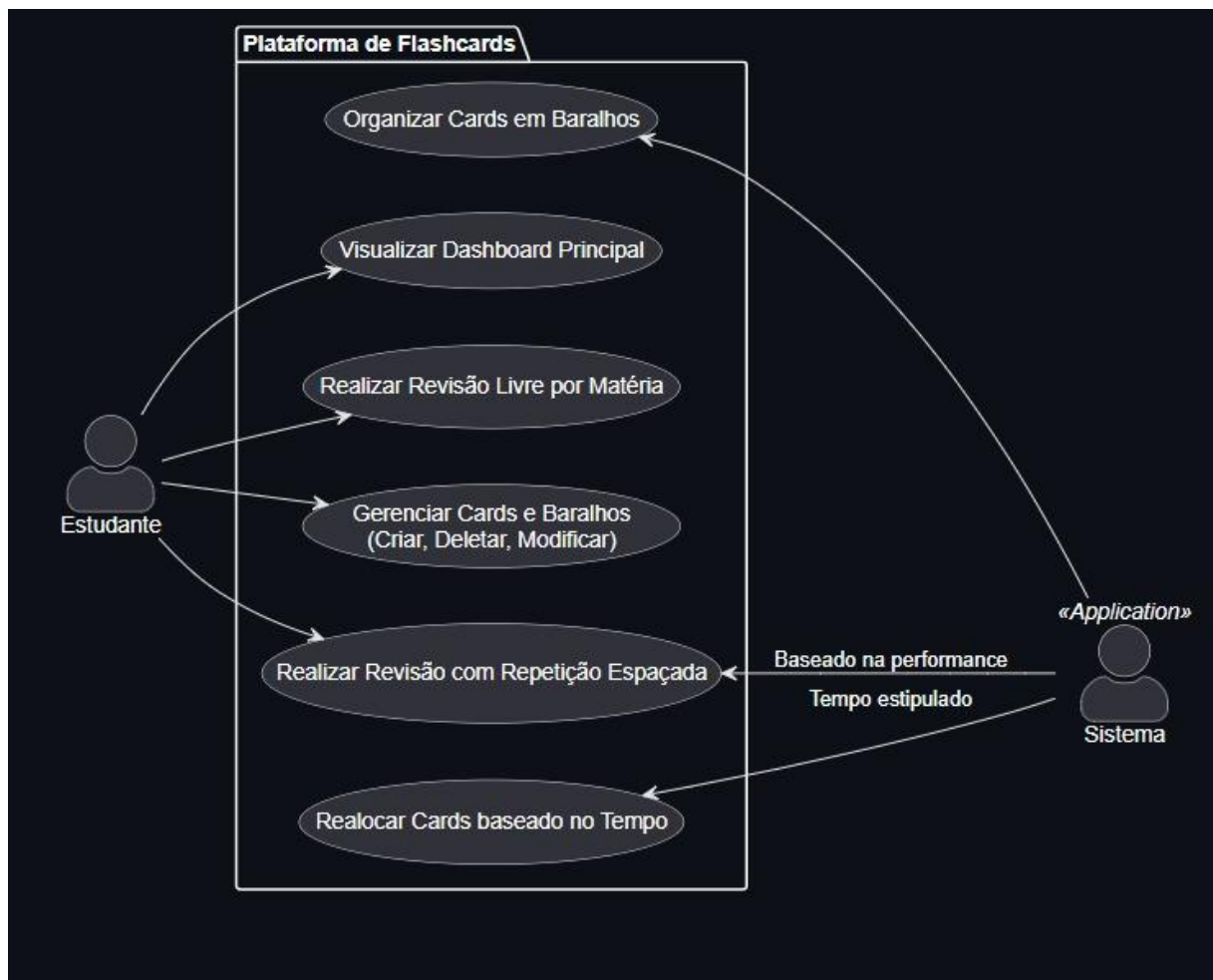
- RNF01 – Usabilidade: A interface deve ser intuitiva e de fácil navegação, permitindo que o usuário utilize o aplicativo sem necessidade de treinamento prévio.
- RNF02 – Segurança: O sistema deve garantir a persistência e a integridade dos dados do usuário de forma local e segura.

- RNF03 – Performance: O aplicativo deve apresentar respostas rápidas às interações do usuário, garantindo baixo consumo de memória e bateria em dispositivos móveis de entrada.
 - RNF04 – Compatibilidade: O sistema deve ser compatível e otimizado primeiramente para o sistema operacional Android, utilizando o framework Flutter.
-

5 MODELAGEM DO SISTEMA

5.1 Casos de Uso

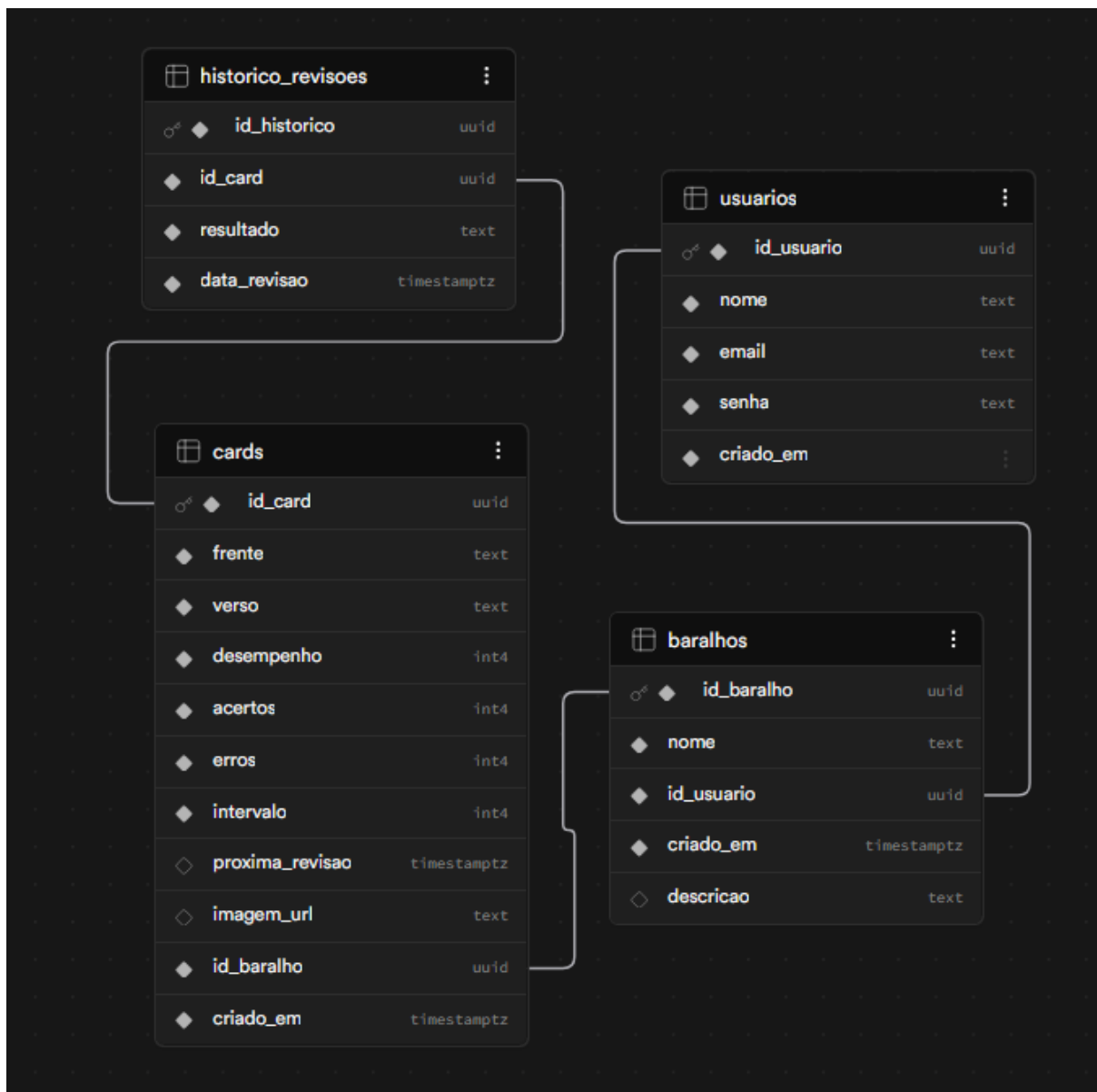
O estudante visualiza o dashboard principal; O estudante realiza a revisão livre por matéria; O estudante gerencia Cards e Baralhos (Criar, deletar, modificar); O sistema organiza cards em baralhos. O estudante realiza revisão com repetição espaçada (baseado na sua performance anterior); O sistema realoca os cards numa seção baseado no tempo estipulado.



5.2 Diagrama de Classes

A estrutura de classes do Quis Notes segue o padrão de persistência definido para o sistema, garantindo a organização dos dados e das funcionalidades de estudo:

- Classe Usuário: Responsável por armazenar as credenciais e informações de perfil do estudante.
- Classe Baralho: Representa o agrupamento temático de conteúdos, vinculando-se a um utilizador específico e contendo múltiplos cartões.
- Classe Card: Contém os atributos fundamentais do flashcard, como o conteúdo da frente (pergunta), o verso (resposta), e os metadados de desempenho utilizados pelo algoritmo de repetição.



5.3 Banco de Dados

O sistema utiliza o Supabase como plataforma de Backend as a Service (BaaS), aproveitando sua infraestrutura baseada em PostgreSQL para a persistência de dados relacionais.

Entidades:

- usuarios (id_usuario, nome, email, senha, criado_em)
- Baralho(id_baralho, nome, id_usuario, criado_em, descricao)
- Card(id, frente, verso, desempenho, id_baralho, acertos, erros, intervalo, proxima_revisao, imagem_url, criado_em)

Relacionamentos:

NA	NB	Cardinalidade	Explicação
Usuário	Baralho	1:N	Um usuário tem vários baralhos.
Baralho	Card	1:N	Um baralho agrupa vários Cards.
Card	Histórico	1:N	Um card é revisado diversas vezes pelo algoritmo.

6 ARQUITETURA DE SOFTWARE

- Padrão arquitetural: Arquitetura Cliente-Servidor baseada em uma API RESTful no backend (Node.js com Express) e o padrão MVC (Model-View-Controller) no cliente mobile (Flutter). O backend adota adicionalmente uma arquitetura em camadas (Layered Architecture: Rotas, Controllers, Services e Model).
- Justificativa da escolha: O desacoplamento entre o cliente mobile e o servidor web simplifica a evolução do sistema. A escolha do TypeScript traz tipagem estática e segurança em tempo de compilação para o ambiente Node.js, reduzindo a ocorrência de erros em produção. O framework Express oferece uma estrutura minimalista e flexível para a criação de rotas HTTP rápidas e eficientes, ideais para o consumo do aplicativo móvel.
- Organização dos componentes:
 - Camada de Apresentação (Client - Flutter): Responsável pela interface do usuário e envio de requisições HTTP/JSON.

- Camada de Roteamento e Controle (API - Express): Recebe as requisições do aplicativo e direciona os dados para as regras de negócio.
 - Camada de Negócio (Services): Onde residem as regras centrais da aplicação, incluindo as validações e o algoritmo de repetição espaçada.
 - Camada de Infraestrutura e Persistência: Faz a comunicação com os serviços do Supabase (Banco de Dados, PostgreSQL).
-

7 PROCESSO DE DESENVOLVIMENTO

Baseado na ISO/IEC 12207:

- Planejamento: Definição das ferramentas necessárias, e funcionalidades do sistema com base na necessidade do usuário. Decisão da arquitetura escolhida e padrões de projeto. Criação das tasks e sprints para início da próxima etapa.
- Desenvolvimento e Testes (Test-Driven Development - TDD): A implementação das regras de negócio em TypeScript e das rotas no Express segue estritamente o ciclo do TDD (Red-Green-Refactor) utilizando o framework Jest:
 - Red: Escreve-se um teste automatizado no Jest para a funcionalidade especificada antes mesmo que o código de produção exista. O teste falha inicialmente.
 - Green: Escreve-se o código de produção em TypeScript com o Express estritamente necessário para fazer o teste automatizado passar com sucesso.
 - Refactor: O código recém-implementado é refatorado para eliminação de redundâncias, aplicação de padrões de projeto e melhoria de legibilidade, garantindo por meio do Jest que nenhuma quebra de comportamento ocorra.
- Integração: Consolidação contínua do código via Git utilizando o fluxo GitFlow. A ramificação principal apenas recebe código que possua 100% de aprovação na suite de testes automatizados do Jest.

- Entrega: Publicação do código no Github, geração de um APK para instalação em dispositivos Android, e apresentação do projeto.
-

8 GARANTIA DA QUALIDADE

8.1 Critérios de Qualidade

Conforme ISO/IEC 25010:

- Manutenibilidade: O uso de TypeScript combinado com a arquitetura em camadas confere alta modularidade e facilidade de extensão ao código de backend.
- Funcionalidade e Confiabilidade: Garantidas pela barreira protetora do TDD, assegurando que modificações futuras na API não quebrem funcionalidades já homologadas.

8.2 Testes

- Testes Unitários Automatizados (Jest): Validação isolada de funções críticas e regras de negócio puras (como o cálculo matemático do intervalo de repetição espaçada no Service), utilizando dados fictícios (mocks) para isolar dependências externas.
 - Testes de Integração Automatizados (Jest/Supertest): Testes que disparam requisições HTTP simuladas contra as rotas do Express, validando o comportamento do fluxo completo de ponta a ponta (controladores, middlewares de segurança e respostas do Supabase).
 - Testes de Usabilidade e Funcionais Manuais: Executados no emulador Android para garantir a experiência final de consumo dos dados da API pelo usuário.
-

9 AVALIAÇÃO DE MATURIDADE

Baseado no CMMI:

- Nível atual do projeto: Nível 1 – Inicial.
- Evidências de maturidade: O processo de desenvolvimento é caracterizado pela agilidade, onde o sucesso do produto depende diretamente do esforço e

da capacidade técnica dos membros da equipe. Contudo, o projeto destaca-se por já adotar práticas avançadas de engenharia de software de forma proativa, como o desenvolvimento orientado a testes (TDD) com Jest e isolamento de dependências.

- Melhorias necessárias: Para que o projeto evolua formalmente em direção ao Nível 2 (Gerenciado), é preciso padronizar os processos de gerência, o que inclui estabelecer métricas quantitativas de cobertura de código nos testes automatizados, monitoramento sistemático de horas por funcionalidade e gerência institucionalizada de defeitos.

10 GERENCIAMENTO DE CONFIGURAÇÃO

O gerenciamento de configuração estabelece os mecanismos necessários para garantir a integridade, o rastreamento e o controle evolutivo de todos os artefatos do ecossistema (código-fonte em Flutter, API em TypeScript e configurações do Supabase):

- Controle de Versão: Utiliza-se o Git como ferramenta de controle de versão distribuído, com repositórios centralizados no GitHub para garantir o armazenamento seguro, histórico de revisões e auditoria do código-fonte.
- Controle de Mudanças: O fluxo de trabalho é baseado na metodologia GitFlow. O desenvolvimento de novas rotas ou telas ocorre em ramificações isoladas (feature branches), que só são integradas à ramificação principal de desenvolvimento após a validação e execução com sucesso de 100% da suíte de testes unitários e de integração automatizados no Jest, além de validação manual.
- Versionamento do Sistema: O sistema por completo (aplicativo móvel e API backend) possui um ciclo de versionamento próprio, independente da documentação, que é gerenciado de forma semântica por meio de *tags* de lançamento diretamente no repositório de código.

11 SEGURANÇA DA INFORMAÇÃO

- O sistema conta com uma API desenvolvida em Node.js utilizando o framework Express, que atua como um gateway de segurança. Esta API é responsável por interceptar as requisições do aplicativo mobile para realizar a regra de negócio da aplicação.
 - No backend Express, são implementados middlewares para validação de esquemas de dados e sanitização de entradas. Além disso, utiliza-se as políticas de Row Level Security (RLS) do Supabase, garantindo que mesmo requisições diretas ao banco de dados respeitem as permissões do usuário autenticado.
 - Toda a troca de informações entre o aplicativo Flutter, a API Node.js e o ecossistema Supabase é protegida pelo protocolo HTTPS, assegurando a criptografia dos dados em trânsito.
-

12 IMPLANTAÇÃO

- O ecossistema completo compreende o aplicativo Android, a API Node.js hospedada em ambiente de nuvem e o banco de dados gerenciado pelo Supabase.
 - O dispositivo móvel deve permitir a instalação de arquivos APK e possuir conexão com a internet para sincronização com a API e o banco de dados.
 - A implantação envolve o deploy do código da API, a configuração das variáveis de ambiente do Supabase e a geração do executável final do aplicativo mobile.
-

13 MANUTENÇÃO

A manutenção do sistema é favorecida pela estrutura modular do Flutter e pela separação de responsabilidades na arquitetura cliente-servidor.

- Tipos de manutenção:

- Corretiva: Consiste no ajuste de bugs e falhas não mapeadas durante a fase de testes, como comportamentos inesperados na interface gráfica do aplicativo ou correções de rotas na API em Express.
- Adaptativa: Engloba a atualização contínua do SDK do Flutter, das dependências do Node.js (como o Jest e o próprio Express) e as adequações necessárias para manter a compatibilidade com atualizações nos serviços de nuvem do Supabase ou novas versões do Android.
- Evolutiva: Foca na expansão do escopo original, adicionando novos recursos (como painéis avançados de estatísticas de estudo) de forma isolada, sem impactar o núcleo da aplicação já implantado.
- Estratégia de atualização: Graças ao desacoplamento entre frontend e backend, muitas melhorias de performance e atualizações em regras de negócio podem ser implantadas no servidor sem interferir no cliente. Isso garante melhorias contínuas sem a necessidade de o usuário instalar uma nova versão do APK constantemente. Atualizações visuais ou novas telas, no entanto, seguirão o fluxo tradicional de distribuição de um novo executável.

14 CONSIDERAÇÕES FINAIS

Resultados obtidos: O desenvolvimento do Quiz Notes alcançou com êxito o objetivo de criar uma ferramenta de estudo ativo otimizada e funcional. A integração de um aplicativo móvel em Flutter com uma API RESTful robusta em TypeScript, validada metodicamente através de TDD, demonstrou a capacidade da equipe em aplicar boas práticas de engenharia de software na construção de um MVP estável e escalável.

Limitações: Por se tratar de um Produto Mínimo Viável focado em uma entrega inicial, o aplicativo depende de conectividade com a internet para autenticação e manipulação dos cartões via Supabase. Além disso, o foco primário de compatibilidade foi estabelecido exclusivamente para o sistema operacional Android.

Trabalhos futuros: Para as próximas iterações do ciclo de vida, planeja-se implementar uma arquitetura *offline-first* utilizando persistência de dados local

combinada com sincronização em segundo plano para o Supabase. Projetam-se também a exportação do sistema para a plataforma iOS, o suporte à anexação de mídias (imagens e áudio) aos flashcards através do *Supabase Storage*, e a criação de funcionalidades colaborativas que permitam o compartilhamento de baralhos entre diferentes usuários.

REFERÊNCIAS

- ISO/IEC 25010 – Systems and software quality models
- ISO/IEC 12207 – Software life cycle processes
- CMMI Institute – Capability Maturity Model Integration
- Guia do Estudante - 3 aplicativos gratuitos para fazer flashcards